# Detecting Derogatory Slur Usage with a Span-Aware Transformer

*Text Mining and Search (TM&S)*

Hrutuja Girish Kargirwar (925351)
Nastaran Janialiabadi (922779)

## 1. Introduction

This project addresses binary classification of derogatory versus non-derogatory usage of slur terms in short, informal text, together with an unsupervised clustering analysis of the same data. The same lexical term (e.g., a slur) can be used to attack, describe neutrally, or be reclaimed by the targeted community, so a classifier that reacts only to the presence of a word rather than its context will systematically misclassify a meaningful share of real-world text.

Two core tasks required by the course are addressed: (1) text classification and (2) text clustering, evaluated on the same dataset and, where relevant, the same learned representations. We compare two text representations (classical TF-IDF and contextualized transformer embeddings) and train and compare four models: two classical linear classifiers, a plain transformer baseline, and a novel span-aware transformer architecture (RAPT) as the project's main contribution.

## 2. Dataset

The Slur Corpus (Kurrek, Saleem & Ruths, 2020, published at the Workshop on Online Abuse and Harms, ACL) was used. It contains 40,003 Reddit comments, each containing exactly one of three slur terms (“tranny”, “nigger”, “faggot”), manually annotated with one of five gold labels:

- DEG — derogatory (20,530 comments)
- NDG — non-derogatory (16,727 comments)
- HOM — homonym / unrelated sense (1,998 comments)
- APR — appropriative/reclaimed usage (553 comments)
- CMP — other (189 comments)

The task was framed as binary DEG vs. non-DEG classification (all other labels grouped). This follows the original authors' own benchmark, allowing comparison against a published external reference rather than only internal baselines. Framing the task as DEG vs. APR only was considered and rejected: APR makes up only 1.4% of the corpus, which would leave a test set with fewer than 100 positive examples and an unreliable F1 estimate.

A stratified 70/15/15 train/dev/test split was used, preserving label balance in every split (52% DEG / 48% non-DEG). A fixed-size random subsample (8,000 rows by default, configurable) was used for training-time efficiency; the pipeline supports running on the full 40,003 rows.

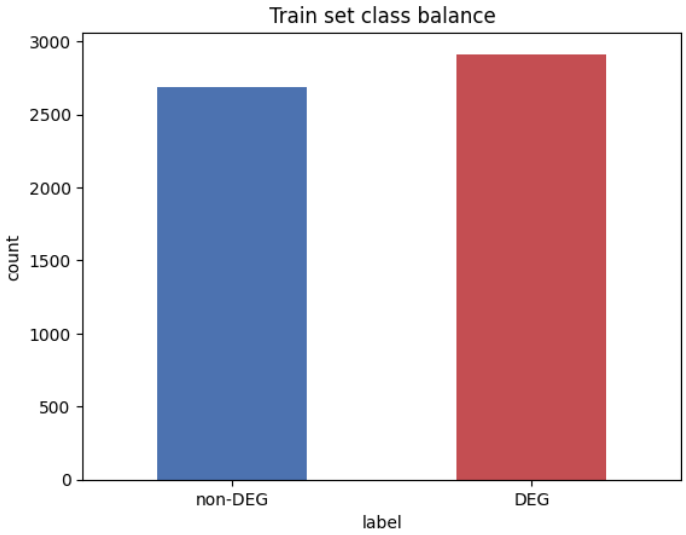



*Figure 1. Class balance of the training split. The stratified split keeps DEG and non-DEG close to even (2,912 vs. 2,688 in this run), so macro-F1 does not need to correct for a skewed baseline.*

## 3. Preprocessing

Two distinct pipelines were used, one per representation — a deliberate choice required by the course brief, which asks that representations not needing full preprocessing be justified explicitly rather than preprocessing everything uniformly.

### 3.1 Shared first step: text normalization

For both pipelines, raw comment text is first normalized: URLs are replaced with a <URL> placeholder token (regex match on http(s):// and www. prefixes), and character elongation is collapsed (any character repeated 3+ times is reduced to 2, e.g. “sooo” → “soo”) to reduce sparse, noisy variants of the same word.

### 3.2 Classical pipeline (for TF-IDF)

Text destined for the TF-IDF representation is further tokenized on word boundaries, lowercased, filtered against scikit-learn's English stopword list (common function words such as “the”, “is”, “and” that carry little discriminative signal for bag-of-words models), and stemmed with the Porter stemmer (reduces inflected words to a common root, e.g. “running”, “ran” → “run”, so that morphological variants are not treated as separate features). This full pipeline is necessary for TF-IDF because it treats each surface word form as an independent feature; without it, sparse and redundant features would dilute the statistical signal and inflate the feature space unnecessarily.

### 3.3 Transformer pipeline (for RAPT / CLS-only baseline)

Text destined for the transformer models is only normalized (Section 3.1) and is explicitly not stemmed or stripped of stopwords. This is because a pretrained transformer already learned subword tokenization and contextual word meaning during large-scale pretraining; removing stopwords or stemming words would destroy exactly the compositional and grammatical information (e.g. negation, function words, word order) that the model relies on to disambiguate meaning from context. Each comment's known slur term (provided directly by the dataset's own `slur` column) is additionally wrapped with [SLUR]...[/SLUR] markers before tokenization, so the span-aware RAPT model (Section 5.3) can locate its exact token position.

## 4. Text Representations

### 4.1 TF-IDF (Term Frequency – Inverse Document Frequency)

Definition: TF-IDF represents each document as a sparse vector over a fixed vocabulary, where each entry weighs how frequent a term is in that document against how rare it is across the whole corpus. For a term t in document d, given a corpus of N documents:

$$TF\text{-}IDF(t, d) = tf(t, d) \times log( N / df(t) )$$

where tf(t, d) is the (sublinear-scaled) count of t in d, and df(t) is the number of documents containing t. Terms that appear in almost every document (e.g. “the”) receive a low weight even if frequent, while terms specific to a smaller subset of documents receive a higher weight. Unigrams and bigrams were used, with the vocabulary capped at 5,000 features by document frequency, and sublinear term-frequency scaling (tf → 1 + log(tf)) applied to dampen the effect of very high raw counts.

Why used: TF-IDF is the standard, computationally cheap baseline representation for text classification, required by the course as a comparison point against denser learned representations, and gives an interpretable lower bound (which words drive which predictions).

### 4.2 Contextualized embeddings (XLM-RoBERTa)

Definition: a contextualized embedding represents each token as a dense vector that depends on the entire surrounding sentence, rather than a fixed, context-independent vector per word type (unlike, e.g., word2vec/GloVe). XLM-RoBERTa (XLM-R) is a transformer encoder pretrained with a masked-language-modelling objective (predicting randomly masked tokens from context) on web text across 100 languages, producing a 768-dimensional embedding per token that reflects both its meaning and its role in that specific sentence.

Why used: contextualized embeddings can, in principle, distinguish “he is a [slur]” from “he is not a [slur]” or “we reclaim the word [slur]” because the representation is computed jointly over the whole sequence, whereas TF-IDF cannot: it would represent all three sentences almost identically since they share nearly the same bag of words. This is the core motivation for using a transformer-based approach for this task.

## 5. Models and Algorithms

### 5.1 Logistic Regression

Definition: a linear classifier that models the probability of the positive class as a sigmoid function of a weighted sum of input features:

$$P(y=1 \mid x) = 1 / (1 + e^{-(w \cdot x + b)})$$

Trained by minimizing binary cross-entropy loss over the training set with respect to weights w and bias b. Applied here on top of the TF-IDF feature vectors (liblinear solver).

Why used / how it helps: a simple, fast, well-regularized linear baseline; because it operates directly on TF-IDF's sparse, high-dimensional feature space, it gives a clear sense of how much signal is present in surface lexical features alone before any deep learning is introduced.

### 5.2 Linear Support Vector Machine (SVM)

Definition: a linear classifier that finds the hyperplane $w \cdot x + b = 0$ maximizing the margin between the two classes, trained by minimizing the hinge loss:

$$L = \lambda \|w\|^2 + \Sigma_i \max(0, 1 - y_i(w \cdot x_i + b))$$

Why used / how it helps: SVMs are known to perform well on sparse, high-dimensional text data (such as TF-IDF vectors) and provide a second, methodologically different classical baseline. Comparing two different algorithms on the same representation (as opposed to only one) satisfies the course's comparative-evaluation requirement more thoroughly and checks that TF-IDF+LR's result is not an artifact of one particular optimizer.

### 5.3 Transformer self-attention (XLM-RoBERTa encoder)

Definition: the core building block of a transformer encoder is scaled dot-product self-attention, which lets every token's representation be updated as a weighted combination of every other token's representation in the same sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}( QK^T / \sqrt{d_k} ) V$$

where Q, K, V (queries, keys, values) are linear projections of the token embeddings and $d_k$ is their dimensionality. Stacking multiple attention heads and feed-forward layers (12 layers in XLM-R-base) lets the model build increasingly abstract, context-dependent representations, including long-range dependencies such as negation scope across several words.

Why used / how it helps: self-attention is precisely the mechanism that gives a transformer access to full-sentence context when representing a single token or the sentence as a whole — the property TF-IDF fundamentally lacks (Section 4.2).

### 5.4 What is [CLS], and the CLS-only baseline

What [CLS] is: every input sequence fed to a BERT-style encoder such as XLM-R is prefixed with a special, non-lexical [CLS] ("classification") token before tokenization even begins. It carries no meaning of its own at the input layer, but because self-attention (Section 5.3) lets every token attend to every other token at every layer, by the final layer the [CLS] position has aggregated information from the entire sequence. Its final-layer embedding is therefore conventionally treated as a single fixed-size summary vector of the whole input, regardless of the input's length.

What the CLS-only baseline is: the standard, conventional way of applying a pretrained transformer to a classification problem. The final-layer [CLS] embedding is passed through a small feed-forward classification head (linear → GELU activation → dropout → linear → softmax) to produce class probabilities; no other part of the sequence is used directly.

Why we use it: it serves as the project's second baseline, deliberately placed between the classical TF-IDF models and RAPT. Its purpose is to isolate two effects that would otherwise be confounded: moving from TF-IDF to this baseline measures the effect of switching to a contextualized representation; moving from this baseline to RAPT (Section 5.5) then measures, in isolation, the additional effect of span-awareness on top of contextualization that is already present. Without this baseline, any improvement from RAPT could not be attributed specifically to its span-fusion mechanism.

### 5.5 What is RAPT, and why we use it

What RAPT is: RAPT (Reclamation/Reappropriation-Aware Processing Transformer) is the custom architecture built for this project, not a pre-existing off-the-shelf model. It reuses the same pretrained XLM-R encoder as the CLS-only baseline, but instead of relying solely on the single [CLS] summary vector, it additionally pools a second, local representation h_slur from only the token span of the marked slur term:

$$h_slur = mean(\, H[slur_start : slur_end] \,), \quad h_cls = H[0]$$

where H is the encoder's full sequence of final-layer token embeddings and slur_start/slur_end are the token positions of the [SLUR]...[/SLUR]-marked term (Section 3.3). The bracket notation follows Python/PyTorch slicing convention exactly as implemented: the interval is half-open, inclusive of slur_start and exclusive of slur_end (H[i, slur_start:slur_end, :] in code), so it spans exactly slur_end − slur_start tokens — by construction, exactly the number of tokens the marked slur term itself tokenizes into (computed as slur_end = slur_start + len(span_token_ids) at preprocessing time), with no off-by-one gap or overlap. h_slur and h_cls are concatenated and passed through the same style of classification head as the baseline:

$$h = [\, h_slur \,;\, h_cls \,] \;\rightarrow\; logits = MLP(h)$$

Why we use it: the motivation is that [CLS] alone, in a long Reddit comment, has to compress the meaning of the entire comment into one vector — the specific way the slur term itself is used locally (e.g. immediately preceded by an insult vs. embedded in a purely descriptive clause) can get diluted among everything else the sentence contains. By explicitly pooling only the marked term's own tokens as a second representation, RAPT gives the classification head a dedicated, focused signal about that specific word's local usage, in addition to (not instead of) the whole-sentence context that h_cls still supplies. This directly reflects the project's task: it is not "is this comment generally hostile" but specifically "is this particular occurrence of this particular slur being used to attack", which is a term-centric question by construction. An optional contrastive loss term (pulling h_slur embeddings of same-label examples together, pushing different-label examples apart within a batch, using a margin-based pairwise loss) was implemented in the codebase to further sharpen h_slur's separation between derogatory and non-derogatory usage, but was not enabled in the final reported run to keep training time within the project's constraints.

#### Implementation note: a truncation bug and its fix

Because Reddit comments are considerably longer than the short tweets such span-aware architectures are usually applied to, the slur span's token position (computed before truncation) could fall past the model's 128-token truncation limit on long comments. Slicing an out-of-range token range produced an empty tensor, and averaging an empty tensor produces NaN, which propagated through the whole network from the first training step, silently collapsing the model to a constant output. The fix falls back to using h_cls in place of h_slur whenever the marked span would be truncated away, which resolved the issue completely and allowed the model to train normally (see Section 6 for the resulting scores).

### 5.6 Optimization: AdamW and cross-entropy loss

All transformer models are trained with cross-entropy loss between the predicted class distribution and the true label, and the AdamW optimizer (Adam with decoupled weight decay), a standard choice for fine-tuning transformers because its adaptive per-parameter learning rates combined with proper weight-decay regularization converge faster and generalize better than plain SGD on this kind of model. Gradient clipping (max norm 1.0) was added as a stability safeguard after the truncation bug above was diagnosed, to bound the effect of any future numerical instability on the model's weights.

## 6. Task 1 — Classification Results

All four models were evaluated on the same held-out test split using macro-averaged F1:

$$F1 = 2 \times (precision \times recall) / (precision + recall), \quad macro\text{-}F1 = mean(F1_class0, F1_class1)$$

Macro-averaging weighs both classes equally regardless of their frequency, which is appropriate here given the near-balanced label distribution (52% DEG / 48% non-DEG).

| Model | Test macro F1 |
|---|---|
| TF-IDF + Logistic Regression | 0.771 |
| TF-IDF + Linear SVM | 0.745 |
| CLS-only Transformer (XLM-R) | 0.872 |
| RAPT (span-aware, main contribution) | 0.875 |

Both transformer-based models clearly outperform the classical TF-IDF baselines by roughly 10 F1 points, confirming that contextualized representations capture the relevant signal (tone, surrounding grammar, intent) better than bag-of-words features for this task. RAPT's span-aware fusion gives a small additional improvement of 0.3 points over the CLS-only baseline, consistent with the hypothesis that explicitly attending to the slur term's local context helps — though the gain is modest relative to what the [CLS] representation alone already captures, indicating that a well-pretrained encoder's global representation is already a very strong signal for this task on its own.

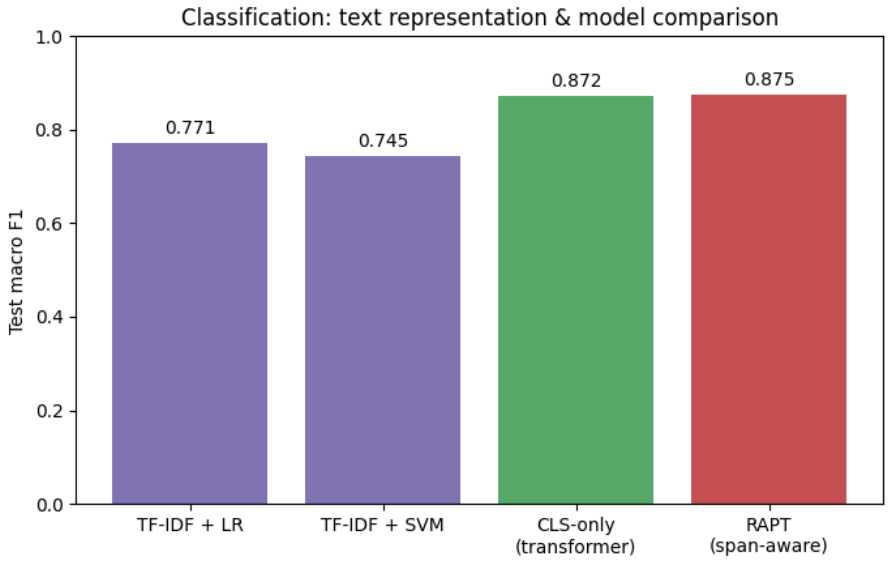



*Figure 2. Test macro-F1 across all four models. The ~10-point jump from the TF-IDF pair to the transformer pair is the largest effect in the whole comparison — representation choice matters far more here than which classifier sits on top of it.*

### 6.1 Training dynamics

Both transformer models were trained for 4 epochs with early checkpoint selection on dev macro-F1 (the best-scoring epoch's weights are the ones evaluated on the test set, not necessarily the last epoch).

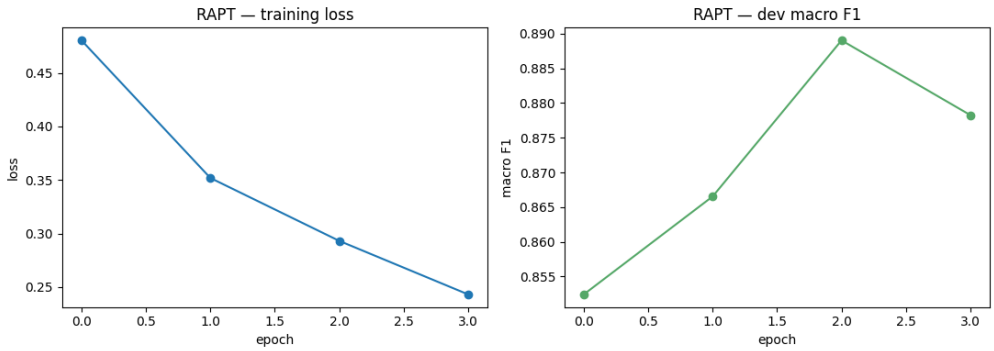



*Figure 3. RAPT training loss and dev macro-F1 per epoch. Loss decreases monotonically (0.48 → 0.24); dev F1 peaks at epoch 3 (0.889) before dipping slightly at epoch 4, a mild sign of the model starting to overfit the small training subsample — this is exactly why the best-epoch checkpoint, not the last one, is kept.*

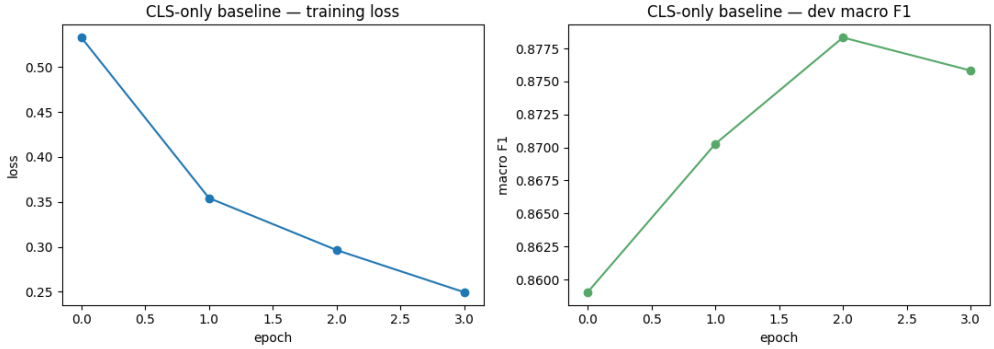



*Figure 4. CLS-only baseline training loss and dev macro-F1 per epoch, for direct visual comparison against Figure 3. The two models follow a very similar training trajectory, which is expected since they share the same underlying encoder and differ only in the classification head's input.*

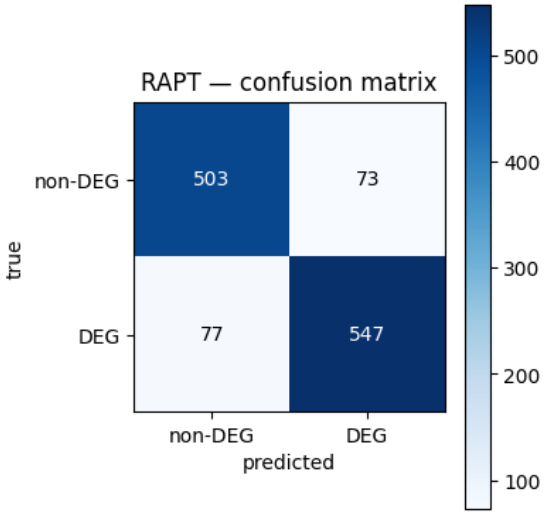


*Figure 5. RAPT confusion matrix on the test set (620 non-DEG, 624 DEG true examples in this run). Errors are close to symmetric (73 false positives, 77 false negatives), indicating no strong bias toward over- or under-predicting either class — consistent with the balanced macro-F1 score.*

## 7. Task 2 — Clustering

### 7.1 K-means

Definition: K-means partitions n points into k clusters by iteratively assigning each point to its nearest centroid and recomputing centroids as the mean of their assigned points, minimizing within-cluster variance:

$$argmin_{\mathsf{m}} \Sigma_{\mathsf{k}} \Sigma_{x \in C_{\mathsf{k}}} \|x - \mu_{\mathsf{k}}\|^2$$

Applied here with k=2 (matching the binary DEG/non-DEG structure) to the RAPT model's [CLS] embeddings extracted from the trained model on the test set (no additional fine-tuning), run with 10 random centroid initializations, keeping the best result.

### 7.2 Evaluation metrics

Adjusted Rand Index (ARI): measures agreement between the unsupervised cluster assignment and the true labels, corrected for chance, ranging from −1 (worse than random) to 1 (perfect agreement), with 0 indicating chance-level agreement.

Silhouette score: for each point, compares its average distance to points in its own cluster against its average distance to points in the nearest other cluster, ranging from −1 to 1; higher values indicate well-separated, well-formed clusters.

### 7.3 Results

- Adjusted Rand Index vs. true labels: 0.501
- Silhouette score: 0.796

An ARI of 0.50 indicates that the unsupervised clusters recover a substantial part of the true label structure without ever seeing the labels during clustering — the embeddings learned during classification training carry a strong, separable derogatory/non-derogatory signal. The high silhouette score (close to 1) confirms the two clusters are well separated in the embedding space, which was also visually confirmed with a 2D PCA projection (Principal Component Analysis, a linear dimensionality-reduction technique that projects high-dimensional data onto the directions of maximum variance for visualization) comparing cluster assignments against the true labels.

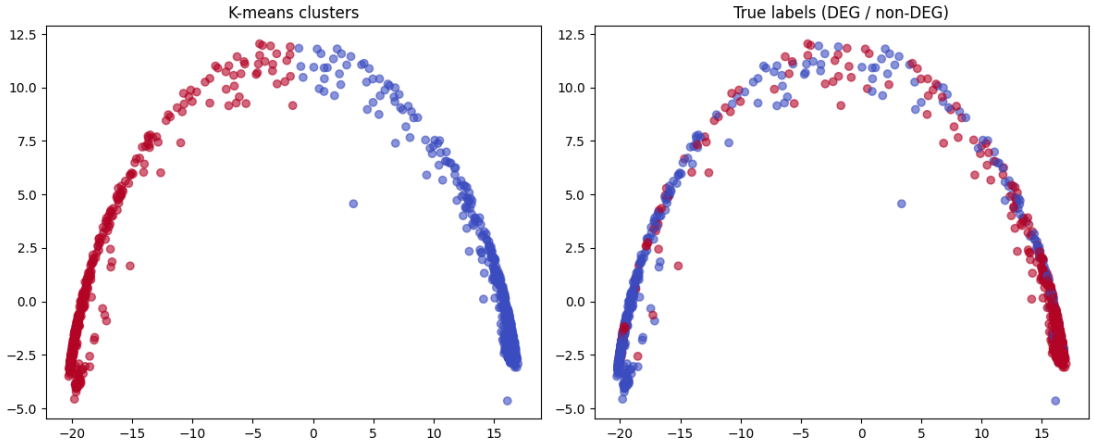



*Figure 6. Left: k-means cluster assignment; right: true DEG/non-DEG labels, both on the same 2D PCA projection of RAPT's [CLS] test-set embeddings. The characteristic arc shape is a well-known artifact of projecting high-dimensional, normalized transformer embeddings down to 2 components — it reflects how PCA distributes variance along the arc rather than a property of the underlying classes. The left/right split along the arc broadly agrees with the true-label split on the right (matching the ARI of 0.50: substantial but imperfect agreement), with the clearest disagreement in the densely mixed middle of the arc, where both classes overlap most.*

## 8. Discussion and Limitations

Manual, qualitative testing with hand-written sentences (a lightweight functional test, in the spirit of the HateCheck methodology of Röttger et al., 2021) revealed a known limitation shared by many hate-speech classifiers: the model correctly flags direct derogatory usage (e.g. “he is a [slur]”) but does not reliably handle negation (e.g. “he is not a [slur]”), tending to key off the presence of the slur term itself rather than fully parsing the surrounding grammatical scope. This is consistent with published findings that hate-speech classifiers frequently learn lexical shortcuts rather than deeper compositional semantics when trained on datasets where negated examples are comparatively rare. Addressing this would require either targeted data augmentation with negated counter-examples or a larger training budget, and is left as future work given the project's time constraints.

The comparative evaluation across four models and two representations (Sections 6–7) nonetheless demonstrates that the chosen architecture and preprocessing choices are sound and that the span-aware fusion provides a measurable, if modest, benefit over a plain contextual baseline.

## 9. Conclusion

This project implemented and compared four models across two text representations for derogatory slur usage detection, and a clustering analysis of the learned representations, on the

Slur Corpus (Kurrek et al., 2020). The main contribution, RAPT, combines a global sentence representation with a local, slur-span-specific representation, and modestly outperforms a plain transformer baseline while both clearly outperform classical TF-IDF baselines. A known limitation around negation handling was identified and documented as a direction for future work.

## References


Kurrek, J., Saleem, H. M., & Ruths, D. (2020). Towards a Comprehensive Taxonomy and Large-Scale Annotated Corpus for Online Slur Usage. Proceedings of the Fourth Workshop on Online Abuse and Harms (WOAH), ACL.

Röttger, P., Vidgen, B., Nguyen, D., Waseem, Z., Margetts, H., & Pierrehumbert, J. (2021). HateCheck: Functional Tests for Hate Speech Detection Models. Proceedings of ACL-IJCNLP 2021.

Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS).

Conneau, A., et al. (2020). Unsupervised Cross-lingual Representation Learning at Scale (XLM-RoBERTa). Proceedings of ACL 2020.